

HEALTHY HABITS TRACKER

Presentation Script — IT111 · Winter 2026 · Team 1

Quantis · Elida · Thaina · Anthony · Sesilina

Due: March 17, 2026 · 10–15 minutes

HOW TO USE THIS SCRIPT

- Read naturally — don't memorize word-for-word. Use this as a guide.
- 💡 TIP boxes are reminders for you, not to be read aloud.
- 🕒 timing targets appear at the top right of each section.
- Keep energy high — speak clearly, make eye contact, own your part.

SLIDE 1 — PROJECT INTRODUCTION

Speaker 1: name

🕒 ~2.5 minutes

Hello everyone. My name is (name), and on behalf of Team 1, welcome to our presentation.

Today we're presenting Healthy Habits Tracker — a full-stack web application built with Python and Flask for our IT111 course here at Seattle Colleges, Winter 2026.

Our team is Quantis, Elida, Thaina, Anthony, and Sesilina.

Let me start with the problem we set out to solve.

Most people have health goals — drink more water, exercise regularly, sleep better, eat cleaner. But the reality is that habits are easy to forget, especially when life gets busy.

We noticed that existing habit tracking apps fall into two categories: either too simple and useless, or so feature-heavy they become overwhelming for students and everyday users.

So we asked ourselves: what if we built something in the middle? A lightweight, focused app that lets you log your daily habits, track your progress over time, and actually see if you're improving.

That's Healthy Habits Tracker. The core idea is three words: Track. Analyze. Improve.

Our target users are students, busy professionals, and anyone trying to build consistent daily routines — without needing a subscription or a complex onboarding process.

💡 TIP: Point to the slide showing the three intro cards when you say 'Track, Analyze, Improve.' Pause briefly after each word.

SLIDE 2 — APPLICATION FEATURES

Speaker 2: name

🕒 ~2.5 minutes

Thank you (name last speaker). My name is (name), and I'll walk you through what our application actually does.

Healthy Habits Tracker has six main features:

First — User Authentication. Users create an account, log in securely, and their data is protected. We implemented a `login_required` decorator in Flask that blocks access to any protected page for unauthenticated users. Sessions persist across requests using Flask's built-in session management.

Second — Multi-Field Habit Logging. This is the heart of the app. Every day, a user can log sleep hours and sleep quality, exercise type — and here's something unique — you can select multiple exercise types in one entry. So if you did a gym session and a run on the same day, both are recorded. Users also log water intake, meals, fruits and vegetables servings, and optional notes.

Third — History Table. Every entry is displayed in a clean, color-coded table. Sleep quality shows as a colored badge — green for great, yellow for okay, red for bad. Exercise shows a checkmark with the activity name. Water cups are color-coded based on whether you hit your goal.

Fourth — Streak Tracking. The app automatically calculates your current streak and your all-time best streak for both exercise and healthy eating. Any exercise type counts — gym, run, walk, yoga, sports — so users feel rewarded for whatever they do.

Fifth — Statistics Dashboard with Chart.js. Users see six KPI cards — total days, exercise days, average water, average sleep, healthy food days, and average exercise minutes. Three interactive charts visualize your last seven entries: a water intake bar chart, a sleep line chart, and a grouped bar chart for exercise and food.

And sixth — Flash Messages. Every action in the app — saving, editing, deleting, errors — gives the user immediate feedback via toast notifications that work consistently across every page.

💡 **TIP:** This is the longest section — keep a steady pace. You can gesture to the laptop screen when mentioning specific features during the demo later.

SLIDE 3 — TECHNICAL ARCHITECTURE

Speaker 3: name

🕒 ~2 minutes

Thank you (name last speaker). I'm (name), and I'll explain how our system is built under the hood.

Our application follows a classic full-stack web architecture with four layers.

At the top — the Browser. This is what the user sees and interacts with: HTML, CSS, and Jinja2 templates that Flask renders dynamically. We used Google Fonts and a fully custom

CSS design — dark background, red and blue accents — to give the app a modern gym-fitness aesthetic.

Below that — Flask in `app.py`. This is our backend. Flask handles all the routing, processes form submissions, manages authentication, and renders templates back to the browser. We have eight routes covering home, log, history, detail view, edit, delete, stats, and authentication.

Below Flask — SQLAlchemy ORM. Instead of writing raw SQL, we defined Python classes called `HabitEntry` and `User`. SQLAlchemy translates those into database operations automatically. This made our code much cleaner and easier to maintain.

At the bottom — SQLite. Our database is a single file called `app.db`, stored in the instance folder. It's created automatically when Flask starts for the first time. SQLite is perfect for this kind of project — lightweight, zero configuration, works everywhere.

One technical decision worth highlighting is the Post/Redirect/Get pattern. When a user submits a habit entry, Flask saves it to the database, then immediately redirects to the history page. This prevents the browser from re-submitting the form if the user refreshes — a common bug in web apps that we solved deliberately.

💡 **TIP:** Point to the architecture diagram on the slide. Say 'Browser → Flask → SQLAlchemy → SQLite' as you gesture left to right across the four boxes.

SLIDE 4 — DATABASE DESIGN & ALGORITHMS

Speaker 4: name

🕒 ~2.5 minutes

Thank you (name last speaker). My name is (name), and I'll cover our database schema and the logic behind the backend algorithms.

We have two database tables.

The User table is simple — ID, username, and password. It handles authentication.

The HabitEntry table is where the real data lives. Every entry has a unique date, and stores twelve fields covering all the habits we track: gym, water, healthy food, sleep hours, sleep quality, exercise type, exercise duration, meals count, fruits and vegetables servings, and notes. One design decision here — exercise type is stored as a comma-separated string. That's how we support multiple exercise types per day in a single database column.

Now let me talk about the algorithms.

The most complex one is Streak Calculation. To find the current streak, we sort all entries by date from newest to oldest, then walk backwards from today. For each day, we check if the user exercised. The moment we hit a gap — a day with no entry — or a rest day, the streak ends. We return the count.

The Best Streak algorithm works differently. We sort entries from oldest to newest and track a running count. Every time two consecutive days both have exercise, the count increments. When we hit a break, the count resets. At the end, we return the highest value ever seen.

The Stats computation is more straightforward — we calculate averages and percentages across all entries: average water, average sleep, exercise percentage, healthy food percentage, and water goal percentage. These numbers feed directly into the six KPI cards on the stats page.

For the charts, we slice the last seven entries from our sorted query and pass the data as JSON to the Jinja2 template, where Chart.js renders it in the browser.

💡 TIP: Speak at a steady pace here — this is the most technical section. If you sense the audience losing attention, emphasize the streak algorithm as the 'interesting' part.

SLIDE 5 — DEVELOPMENT PROCESS & CHALLENGES

Speaker 4: name (continues)

🕒 ~1.5 minutes

Now let me share the challenges we ran into during development, and how we solved them.

Our first bug was a SQLite date type error. When we submitted the log form, Flask was passing the date as a plain string — but SQLAlchemy requires a native Python date object. We fixed it by converting the string with `date_type.fromisoformat()` before creating the database entry.

Second — our navigation was breaking. The Flask `url_for()` function was raising `BuildError` exceptions because teammates had different function names in their branches. We solved it by replacing `url_for()` calls in the navbar with direct URL paths — slash history, slash log — which work regardless of function names.

Third — Chart.js wasn't rendering. We were loading the CDN script tag in the HTML, then immediately calling `initCharts()`. The problem was that the CDN hadn't finished loading yet. We fixed it by injecting the script tag dynamically in JavaScript with an `onload` callback, so `initCharts()` only runs after the library is fully loaded.

Fourth — CSS was breaking the page layout. We had style blocks inside the Jinja content block, which caused the navbar to render in the middle of the page on some templates. The fix was to add a dedicated styles block inside the HTML head in `base.html`, and move all page-specific CSS there.

Each of these bugs taught us something real — about how Flask handles data types, how JavaScript loading works, and how Jinja template inheritance should be structured.

💡 TIP: Own these bugs confidently — they show you understand the full stack. Saying 'we ran into this and here's exactly how we fixed it' is impressive to any technical audience.

SLIDE 6 — LIVE DEMO

Speaker 5: Name

 ~4 minutes

Thank you (name last speaker). I'm (name), and I'll walk you through the application live.

Step 1 — Home Page

We start on the home page. You can see the dark gym-inspired design — the hero section with the barbell mascot, and the three feature cards explaining what the app tracks. The navbar gives us access to every section of the app.

Step 2 — Sign Up

Let me create a new account. I'll enter a username and password. When I click create account, Flask validates the input, stores the user in the database, starts a session, and redirects back home with a success flash message — you can see it at the top of the screen.

Step 3 — Log Habits

Now I'll go to Log Habits. Notice the form design — sleep quality has three pill buttons for great, okay, and bad. Exercise type uses checkboxes so I can select multiple options — I'll pick Gym and Run. Water and other numeric fields use plus/minus steppers. I'll fill in today's data and save. The app redirects to History — that's the Post/Redirect/Get pattern working correctly.

Step 4 — History Table

Here's the History page. Each row shows the sleep badge color-coded by quality, the exercise checkmark with the activity types listed, nutrition with servings count, and water cups in blue when the goal is met. Every entry has View, Edit, and Delete buttons.

Step 5 — Stats Page

Now the Stats page. At the top you see the six KPI cards — total days tracked, exercise days with percentage, average water, average sleep, healthy food days, and average exercise minutes. Below that, the streak cards show our current and best streak — the fire emoji lights up when you're on a three-day or longer streak. And then the three Chart.js charts — water bar, sleep line, and exercise versus healthy food side by side. At the bottom, three doughnut rings show overall consistency percentages.

Step 6 — Detail View

If I click View on any entry, I get a full detail page. Three cards summarize gym, water, and food. Below that, a score bar shows how many of five daily goals were hit. If there are notes, they appear at the bottom. From here I can edit or delete the entry.

 **TIP:** Walk through this slowly. Don't rush. If something goes wrong — stay calm, go to the next step. Screenshots are ready as backup.

BACKUP PLAN: If the app fails to start, open the screenshots folder and show them in order. Say: 'Let me show you the screenshots I prepared as a backup.'

SLIDE 7 — CONCLUSION

Speaker 5: name (continues)

🕒 ~1.5 minutes

To wrap up — let me share what this project meant for us.

We built a production-quality web application from scratch. Not just a basic CRUD app — we added authentication, streak algorithms, Chart.js data visualization, and a fully custom dark UI theme. We delivered more than the MVP scope required.

We solved real engineering bugs — date type mismatches, async JavaScript loading, CSS template inheritance, and routing conflicts. Each one made us better developers.

We practiced real team collaboration with Git — branches, pull requests, merge conflicts, and code reviews. That workflow mirrors what professional software teams do every day.

And we learned that full-stack development is about connecting many layers — frontend design, backend logic, database schema, and deployment considerations — all working together.

If we had more time, we'd add password hashing with bcrypt, export to CSV, PostgreSQL for production, and eventually deploy to the cloud with features like habit reminders and goal setting.

Our repository is available on GitHub at github.com/QuantisX/Team-1-s-Web-App.

Thank you very much. We're happy to take any questions.

💡 **TIP:** Smile. Pause after 'thank you'. Let the silence land. Don't immediately ask 'any questions?' — wait two seconds first.

TIMING SUMMARY

Section	Speaker	Time
Project Introduction	Speaker 1	~2.5 min
Application Features	Speaker 2	~2.5 min
Technical Architecture	Speaker 3	~2 min
DB Design & Algorithms	Speaker 4	~2.5 min
Development Challenges	Speaker 4	~1.5 min
Live Demo	Speaker 5	~4 min
Conclusion & Q&A	Speaker 5	~1.5 min
TOTAL	All	~16 min

Note: 16 minutes is intentional — presentations often run shorter than rehearsed. If you finish at 13-14 minutes, that's perfect.

QUICK TIPS FOR PRESENTATION DAY

BEFORE CLASS

- Test Flask runs: `python -m flask --app healthy_habits_tracker/app.py run`
- Seed the database with at least 5-7 real entries
- Have screenshots ready in a folder as backup
- Close all unnecessary apps — browser tabs, Slack, email
- Open VS Code to `app.py` in case you want to show the code

DURING THE PRESENTATION

- Speak to the audience, not the screen
- Slow down when showing the demo — don't rush
- If something breaks, stay calm and move to screenshots
- Each person should introduce themselves before speaking
- Own your bugs — they show technical depth, not weakness

QUESTIONS TO PREPARE FOR

- Why SQLite and not PostgreSQL? → SQLite is ideal for MVP/local development. PostgreSQL would be the next step for production.
- Why no password hashing? → We used plain text for the MVP to focus on features. Bcrypt is listed as a future improvement.
- How does the streak algorithm handle timezones? → We compare Python date objects, which are timezone-naive. For production, we'd add UTC handling.
- Could this be deployed? → Yes — Render or Railway would work with a Postgres swap and a `requirements.txt` update.